

Vertex input description

- [Introduction](#)
- [Vertex shader](#)
- [Vertex data](#)
- [Binding descriptions](#)
- [Attribute descriptions](#)
- [Pipeline vertex input](#)

Introduction

In the next few chapters, we're going to replace the hardcoded vertex data in the vertex shader with a vertex buffer in memory. We'll start with the easiest approach of creating a CPU visible buffer and using `memcpy` to copy the vertex data into it directly, and after that we'll see how to use a staging buffer to copy the vertex data to high performance memory.

Vertex shader

First change the vertex shader to no longer include the vertex data in the shader code itself. The vertex shader takes input from a vertex buffer using the `in` keyword.

```
#version 450

layout(location = 0) in vec2 inPosition;
layout(location = 1) in vec3 inColor;

layout(location = 0) out vec3 fragColor;

void main() {
    gl_Position = vec4(inPosition, 0.0, 1.0);
    fragColor = inColor;
}
```

The `inPosition` and `inColor` variables are *vertex attributes*. They're properties that are specified per-vertex in the vertex buffer, just like we manually specified a position and color per vertex using the two arrays. Make sure to recompile the vertex shader!

Just like `fragColor`, the `layout(location = x)` annotations assign indices to the inputs that we can later use to reference them. It is important to know that some types, like `dvec3` 64 bit vectors, use multiple *slots*. That means that the index after it must be at least 2 higher:

```
layout(location = 0) in dvec3 inPosition;
layout(location = 2) in vec3 inColor;
```

You can find more info about the layout qualifier in the [OpenGL wiki](https://www.khronos.org/opengl/) (<https://www.khronos.org/opengl/>)

[wiki/Layout_Qualifier_\(GLSL\)](#)).

Vertex data

We're moving the vertex data from the shader code to an array in the code of our program. Start by including the GLM library, which provides us with linear algebra related types like vectors and matrices. We're going to use these types to specify the position and color vectors.

```
#include <glm/glm.hpp>
```

Create a new structure called `Vertex` with the two attributes that we're going to use in the vertex shader inside it:

```
struct Vertex {  
    glm::vec2 pos;  
    glm::vec3 color;  
};
```

GLM conveniently provides us with C++ types that exactly match the vector types used in the shader language.

```
const std::vector<Vertex> vertices = {  
    {{0.0f, -0.5f}, {1.0f, 0.0f, 0.0f}},  
    {{0.5f, 0.5f}, {0.0f, 1.0f, 0.0f}},  
    {{-0.5f, 0.5f}, {0.0f, 0.0f, 1.0f}}  
};
```

Now use the `Vertex` structure to specify an array of vertex data. We're using exactly the same position and color values as before, but now they're combined into one array of vertices. This is known as *interleaving* vertex attributes.

Binding descriptions

The next step is to tell Vulkan how to pass this data format to the vertex shader once it's been uploaded into GPU memory. There are two types of structures needed to convey this information.

The first structure is `VkVertexInputBindingDescription` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/VkVertexInputBindingDescription.html>) and we'll add a member function to the `Vertex` struct to populate it with the right data.

```

struct Vertex {
    glm::vec2 pos;
    glm::vec3 color;

    static VkVertexInputBindingDescription getBindingDescription() {
        VkVertexInputBindingDescription bindingDescription{};

        return bindingDescription;
    }
};

```

A vertex binding describes at which rate to load data from memory throughout the vertices. It specifies the number of bytes between data entries and whether to move to the next data entry after each vertex or after each instance.

```

VkVertexInputBindingDescription bindingDescription{};
bindingDescription.binding = 0;
bindingDescription.stride = sizeof(Vertex);
bindingDescription.inputRate = VK_VERTEX_INPUT_RATE_VERTEX;

```

All of our per-vertex data is packed together in one array, so we're only going to have one binding. The `binding` parameter specifies the index of the binding in the array of bindings. The `stride` parameter specifies the number of bytes from one entry to the next, and the `inputRate` parameter can have one of the following values:

- `VK_VERTEX_INPUT_RATE_VERTEX`: Move to the next data entry after each vertex
- `VK_VERTEX_INPUT_RATE_INSTANCE`: Move to the next data entry after each instance

We're not going to use instanced rendering, so we'll stick to per-vertex data.

Attribute descriptions

The second structure that describes how to handle vertex input is `VkVertexInputAttributeDescription` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/VkVertexInputAttributeDescription.html>). We're going to add another helper function to `Vertex` to fill in these structs.

```

#include <array>

...

static std::array<VkVertexInputAttributeDescription, 2> getAttributeDescriptions() {
    std::array<VkVertexInputAttributeDescription, 2> attributeDescriptions{};

    return attributeDescriptions;
}

```

As the function prototype indicates, there are going to be two of these structures. An attribute description struct describes how to extract a vertex attribute from a chunk of vertex data originating from a binding

description. We have two attributes, position and color, so we need two attribute description structs.

```
attributeDescriptions[0].binding = 0;
attributeDescriptions[0].location = 0;
attributeDescriptions[0].format = VK_FORMAT_R32G32_SFLOAT;
attributeDescriptions[0].offset = offsetof(Vertex, pos);
```

The `binding` parameter tells Vulkan from which binding the per-vertex data comes. The `location` parameter references the `location` directive of the input in the vertex shader. The input in the vertex shader with location `0` is the position, which has two 32-bit float components.

The `format` parameter describes the type of data for the attribute. A bit confusingly, the formats are specified using the same enumeration as color formats. The following shader types and formats are commonly used together:

- `float`: `VK_FORMAT_R32_SFLOAT`
- `vec2`: `VK_FORMAT_R32G32_SFLOAT`
- `vec3`: `VK_FORMAT_R32G32B32_SFLOAT`
- `vec4`: `VK_FORMAT_R32G32B32A32_SFLOAT`

As you can see, you should use the format where the amount of color channels matches the number of components in the shader data type. It is allowed to use more channels than the number of components in the shader, but they will be silently discarded. If the number of channels is lower than the number of components, then the BGA components will use default values of `(0, 0, 1)`. The color type (`SFLOAT`, `UINT`, `SINT`) and bit width should also match the type of the shader input. See the following examples:

- `ivec2`: `VK_FORMAT_R32G32_SINT`, a 2-component vector of 32-bit signed integers
- `uvec4`: `VK_FORMAT_R32G32B32A32_UINT`, a 4-component vector of 32-bit unsigned integers
- `double`: `VK_FORMAT_R64_SFLOAT`, a double-precision (64-bit) float

The `format` parameter implicitly defines the byte size of attribute data and the `offset` parameter specifies the number of bytes since the start of the per-vertex data to read from. The binding is loading one `Vertex` at a time and the position attribute (`pos`) is at an offset of `0` bytes from the beginning of this struct. This is automatically calculated using the `offsetof` macro.

```
attributeDescriptions[1].binding = 0;
attributeDescriptions[1].location = 1;
attributeDescriptions[1].format = VK_FORMAT_R32G32B32_SFLOAT;
attributeDescriptions[1].offset = offsetof(Vertex, color);
```

The color attribute is described in much the same way.

Pipeline vertex input

We now need to set up the graphics pipeline to accept vertex data in this format by referencing the structures in `createGraphicsPipeline`. Find the `vertexInputInfo` struct and modify it to reference the two descriptions:

```
auto bindingDescription = Vertex::getBindingDescription();
auto attributeDescriptions = Vertex::getAttributeDescriptions();

vertexInputInfo.vertexBindingDescriptionCount = 1;
vertexInputInfo.vertexAttributeDescriptionCount = static_cast<uint32_t>(attributeDescriptions.size());
vertexInputInfo.pVertexBindingDescriptions = &bindingDescription;
vertexInputInfo.pVertexAttributeDescriptions = attributeDescriptions.data();
```

The pipeline is now ready to accept vertex data in the format of the `vertices` container and pass it on to our vertex shader. If you run the program now with validation layers enabled, you'll see that it complains that there is no vertex buffer bound to the binding. The next step is to create a vertex buffer and move the vertex data to it so the GPU is able to access it.

[C++ code \(../code/18_vertex_input.cpp\)](#) / [Vertex shader \(../code/18_shader_vertexbuffer.vert\)](#) / [Fragment shader \(../code/18_shader_vertexbuffer.frag\)](#)

[Previous \(/en/Drawing_a_triangle/Swap_chain_recreation\)](#)

[Next \(/en/Vertex_buffers/Vertex_buffer_creation\)](#)

ALSO ON VULKAN TUTORIAL

Image view and sampler

10 years ago · 30 comments

A tutorial that teaches you everything it takes to render 3D graphics with ...

Swap chain recreation

10 years ago · 219 comments

A tutorial that teaches you everything it takes to render 3D graphics with ...

Rendering and presentation

10 years ago · 424 comments

A tutorial that teaches you everything it takes to render 3D graphics with ...

Vulkan T

10 years ag

A tutorial t everything 3D graphic